

Lab 04 — Praktijklab: de image van de API bouwen

Doel: zelf de Dockerfile van een (nep-)Spring Boot-API schrijven, en elke valkuil uit de cursus uitlokken — context, cache, `CMD` / `ENTRYPOINT`, shell-vorm, `USER` — met een build-engine die geen daemon heeft.

Vereisten — Labs 01 tot 03 afgewerkt.

Geleverde bestanden (`files/`) - `Api.java` — een HTTP-API van 30 regels, zonder afhankelijkheden. Ze stelt `/` en `/actuator/health` bloot, leest `APP_MESSAGE`, `APP_PROFILE`, `SERVER_PORT` uit de omgeving, en behandelt `SIGTERM`. **Je zult ze nooit hoeven te wijzigen:** de labs gaan over containers, niet over Java. - `construire-jar.sh` — compileert `Api.java` naar `api.jar` in een wegwerptainer, zonder een JDK op je WSL te installeren.

Stap 1 — Het project voorbereiden

```
mkdir -p ~/labo-docker/04 && cd ~/labo-docker/04
cp <pad-van-het-lab>/files/Api.java .
cp <pad-van-het-lab>/files/construire-jar.sh . && chmod +x construire-jar.sh
./construire-jar.sh
```

Observeer de download van `docker.io/library/eclipse-temurin:21-jdk` (één keer, ~490 MB), en dan `api.jar` van ongeveer 2,4 KB.

Uitleg. Je hebt zonet een container gebruikt als **wegwerptool**: de compilatie gebeurde in een volledige JDK, gemount op je map, en er blijft niets van over. Dat is al het idee van de multi-stage van lab 05.

→ JAVA

`javac` compileert een `.java` naar `.class` (bytecode, onafhankelijk van de machine); `jar` verpakt de `.class`-bestanden in een ZIP-archief met een manifest dat zegt welke klasse te starten. De **JDK** (Development Kit) bevat die tools; de **JRE** (Runtime Environment) bevat alleen wat nodig is om *uit te voeren* — vandaar zijn half zo grote omvang, en de keuze die we voor de uiteindelijke image zullen maken.

Stap 2 — Een eerste Dockerfile

Maak `Dockerfile` aan:

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY api.jar /app/api.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
CMD ["--spring.profiles.active=prod"]
```

```
podman build -t api-lab:1.0 .
podman images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}' | grep -E 'api-lab|temurin'
```

Observeer de stappen STEP 1/6 tot STEP 6/6, elk gevolgd door een identificatie --> ..., en dan COMMIT api-lab:1.0 en Successfully tagged localhost/api-lab:1.0. De image weegt **209 MB** — voor een JAR van 2,4 KB. Het grootste deel is de JRE.

```
podman run -d --name api -p 18080:8080 api-lab:1.0
curl -s localhost:18080/ ; echo
curl -s localhost:18080/actuator/health ; echo
podman logs api
podman port api
```

Observeer {"message":"Bonjour depuis l'API","profile":"default"}, {"status":"UP"}, de regels Arguments recus : --spring.profiles.active=prod en API demarree sur le port 8080 (profil default), en 8080/tcp -> 0.0.0.0:18080.

Uitleg. EXPOSE 8080 heeft niets gepubliceerd: -p 18080:8080 heeft de doorsturing aangemaakt. Ga het na: podman run -d --name api2 api-lab:1.0 en dan curl -m 2 localhost:18080/ faalt... en onthoud dat in rootless-modus een -p 80:8080 geweigerd zou worden.

```
podman rm -f -t 0 api api2
```

→ WINDOWS / WSL

Open <http://localhost:18080/actuator/health> in je Windows-browser: WSL stuurt de poort door. Zo zul je in lab 07 de Angular-frontend testen.

Stap 3 — De build context

```
mkdir -p ../gemeenschappelijk && echo "privesleutel" > ../gemeenschappelijk/secret.txt
printf 'FROM docker.io/library/alpine\nCOPY ../gemeenschappelijk/secret.txt /\n' >
Dockerfile.buiten-context
podman build -f Dockerfile.buiten-context -t poging .
```

Observeer de mislukking: Error: building at STEP "COPY ../gemeenschappelijk/secret.txt /": ... possible escaping context directory error: copier: stat: "/gemeenschappelijk/secret.txt": no such file or directory.

Uitleg. Buildah heeft het pad *binnen* de context teruggebracht (/gemeenschappelijk/secret.txt relatief ten opzichte van de map) en er niets gevonden. Het is geen rechtenprobleem: het bestand ligt buiten de perimeter.

Meet nu de kost van een ongefilterde context:

```
mkdir -p node_modules && dd if=/dev/zero of=node_modules/groot.bin bs=1M count=200 2>/dev/null
printf 'FROM docker.io/library/alpine\nCOPY . /src\n' > Dockerfile.alles
time podman build -q -f Dockerfile.alles -t api-lab:alles .
podman images --format '{{.Repository}}:{{.Tag}} {{.Size}}' | grep alles
```

Observeer een build van ongeveer 1,7 s... en een image van **218 MB** voor een `alpine` van 8,7 MB: de 200 MB van `node_modules` zitten erin.

```
printf 'node_modules\n*.bin\nDockerfile.buiten-context\n' > .dockerignore
time podman build -q -f Dockerfile.alles -t api-lab:alles2 .
podman images --format '{{.Repository}}:{{.Tag}} {{.Size}}' | grep alles
```

Observeer `8.71 MB` voor `alles2`, en een snellere build.

Uitleg. Met Docker zouden die 200 MB eerst naar de daemon **overgedragen** zijn (`transferring context`); Podman leest de map ter plaatse, de build lijkt dus "snel". Maar de image neemt alles mee wat de `COPY . .` aanraakt. `.dockerignore` werkt **vóór** elke instructie: zonder zou `node_modules` in de gepubliceerde image belanden. Podman aanvaardt ook de naam `.containerignore` — zelfde inhoud, zelfde effect.

Stap 4 — De cache, en de volgorde van de instructies

Voeg een gesimuleerde stap "afhankelijkheden" toe. Vervang je `Dockerfile` door:

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
RUN echo "afhankelijkheden downloaden..." && sleep 5
COPY api.jar /app/api.jar
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
```

```
time podman build -t api-lab:2.0 .
time podman build -t api-lab:2.0 .
```

Observeer een eerste build van ongeveer 6,4 seconden, en een tweede in 0,7 s, met `--> Using cache` onder elke stap.

Wijzig de JAR en bouw opnieuw:

```
touch Api.java && ./construire-jar.sh >/dev/null
time podman build -t api-lab:2.1 .
```

Observeer dat de stap `RUN ... sleep 5` nog altijd `--> Using cache` toont: alleen de `COPY` en wat volgt worden opnieuw gespeeld. 0,7 s.

Keer nu de volgorde om — de `COPY` **vóór** de `RUN`:

```
FROM docker.io/library/eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY api.jar /app/api.jar
RUN echo "afhankelijkheden downloaden..." && sleep 5
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
```

```
podman build -t api-lab:3.0 .
./construire-jar.sh >/dev/null && time podman build -t api-lab:3.1 .
```

Observeer dat de 5 seconden **elke keer betaald** worden: 6,3 s.

Uitleg. Ziedaar, in het klein, het verschil tussen een Maven-build van 40 seconden en een van 6 minuten: de vluchtige `COPY` vóór de dure stap maakt alles wat volgt ongeldig.

Stap 5 — `CMD` tegenover `ENTRYPOINT`

Bouw de image van stap 2 opnieuw (ze heeft een `ENTRYPOINT` **en** een `CMD`) en observeer de combinatie:

```
timeout 5 podman run --rm api-lab:1.0 | head -2
timeout 5 podman run --rm api-lab:1.0 --debug | head -2
```

Observeer in het eerste geval `Arguments recurs : --spring.profiles.active=prod`, in het tweede `Arguments recurs : --debug`. De `ENTRYPOINT` (`java -jar ...`) is gebleven; alleen de `CMD` is vervangen.

Vergelijk met een image die alleen een `CMD` heeft:

```
printf 'FROM docker.io/library/eclipse-temurin:21-jre-alpine\nCOPY api.jar /app/api.jar\nCMD ["java","-jar","/app/api.jar"]\n' > D-cmd
podman build -q -f D-cmd -t api-lab:cmd .
podman run --rm api-lab:cmd sh -c 'echo "ik vervang alles"'
```

Observeer dat de API **helemaal niet** start: met een `CMD` alleen vervangt het argument het hele commando.

```
podman run --rm --entrypoint sh api-lab:1.0 -c 'echo "shell verkregen ondanks ENTRYPOINT"'
```

Observeer dat `--entrypoint` de nooduitgang is om te debuggen.

Uitleg. `CMD` = vervangbare standaardwaarde, `ENTRYPOINT` = vast programma waaraan argumenten worden toegevoegd. Het bedrijfs patroon is `ENTRYPOINT` + `CMD` voor de standaardargumenten.

Stap 6 — De *shell*-vorm, of de kapotte stop

Dit is het belangrijkste experiment van het lab. Bouw **dezelfde** applicatie met de *shell*-vorm, op een **Ubuntu**-basis (de image `21-jre` zonder achtervoegsel):

```
printf 'FROM docker.io/library/eclipse-temurin:21-jre\nCOPY api.jar /app/api.jar\nENTRYPOINT java -jar /app/api.jar\n' > D-shell-debian
podman build -q -f D-shell-debian -t api-lab:shell-debian .
podman run -d --name s-deb api-lab:shell-debian
sleep 3
podman exec s-deb ps -o pid,args | head -3
```

Observeer:

```
PID COMMAND
1 /bin/sh -c java -jar /app/api.jar
2 java -jar /app/api.jar
```

De shell is PID 1 gebleven. Stop:

```
time podman stop s-deb
podman inspect --format 'code={{.State.ExitCode}}' s-deb
podman logs s-deb | tail -2
```

Observeer de waarschuwing `resorting to SIGKILL`, `real 0m10.8s`, `code=137`, en **geen** boodschap "SIGTERM reçu": de *shutdown hooks* zijn niet uitgevoerd.

Vergelijk met de `exec`-vorm van stap 2:

```
podman run -d --name s-exec api-lab:1.0
sleep 3 ; podman exec s-exec ps -o pid,args | head -3
time podman stop s-exec
podman inspect --format 'code={{.State.ExitCode}}' s-exec
podman logs s-exec | tail -2
```

Observeer `1 java -jar /app/api.jar --spring.profiles.active=prod`, een stop in **0,14 s**, `code=143`, en de regels `SIGTERM reçu : arret propre en cours...` en `API arretee proprement`.

Ten slotte dezelfde *shell*-vorm maar op **Alpine**-basis:

```
printf 'FROM docker.io/library/eclipse-temurin:21-jre-alpine\nCOPY api.jar\n/app/api.jar\nENTRYPOINT java -jar /app/api.jar\n' > D-shell-alpine
podman build -q -f D-shell-alpine -t api-lab:shell-alpine .
podman run -d --name s-alp api-lab:shell-alpine ; sleep 3
podman exec s-alp ps -o pid,args | head -3
time podman stop s-alp ; podman inspect --format 'code={{.State.ExitCode}}' s-alp
```

Observeer dat Java deze keer **wel** PID 1 is en dat de stop netjes verloopt (`143`).

Uitleg. De shell van busybox (Alpine) vervangt zichzelf door het commando wanneer dat eenvoudig is; de `dash` van Ubuntu doet dat niet. **Dezelfde Dockerfile heeft dus twee gedragingen naargelang de basisimage.** Dat is precies het soort bug dat werkt op de werkpost van de ontwikkelaar en breekt in productie. De `exec`-vorm neemt de vraag weg.

```
podman rm s-deb s-exec s-alp
```

Stap 7 — Het opstartscript zonder `exec`

Het meest voorkomende geval in bedrijven:

```
printf '#!/bin/sh\necho "voorbereiding..."&#x2D;java -jar /app/api.jar\n' > entrypoint.sh
chmod +x entrypoint.sh
printf 'FROM docker.io/library/eclipse-temurin:21-jre-alpine\nCOPY api.jar /app/api.jar\nCOPY
entrypoint.sh /entrypoint.sh\nENTRYPOINT ["/entrypoint.sh"]\n' > D-script
podman build -q -f D-script -t api-lab:script .
podman run -d --name s-script api-lab:script ; sleep 3
podman exec s-script ps -o pid,args | head -4
time podman stop s-script
podman inspect --format 'code={{.State.ExitCode}}' s-script
```

Observeer 1 {entrypoint.sh} /bin/sh /entrypoint.sh en 2 java -jar /app/api.jar, en dan de 10 seconden en code 137 — **zelfs op Alpine**.

Corrigeer door `exec` toe te voegen:

```
printf '#!/bin/sh\nexec java -jar /app/api.jar\n' > entrypoint.sh
podman build -q -f D-script -t api-lab:script2 .
podman rm -f -t 0 s-script && podman run -d --name s-script api-lab:script2 ; sleep 3
podman exec s-script ps -o pid,args | head -3
time podman stop s-script ; podman inspect --format 'code={{.State.ExitCode}}' s-script
podman rm s-script
```

Observeer dat er geen shell meer is, dat de stop onmiddellijk is en dat de code 143 is.

Stap 8 — ARG is geen kluis

```
printf 'FROM docker.io/library/alpine\nARG DB_PASSWORD=leeg\nRUN echo "build met $DB_PASSWORD"
> /trace.txt\nCMD ["cat","/trace.txt"]\n' > D-arg
podman build -q -f D-arg --build-arg DB_PASSWORD='Secr3t!' -t api-lab:arg .
podman image inspect --format '{{json .Config.Env}}' api-lab:arg
podman history --no-trunc api-lab:arg --format '{{.CreatedBy}}' | head -3
```

Observeer dat `Config.Env` alleen `PATH` bevat (klopt, `ARG` wordt geen `ENV`)... maar dat `podman history` |1 DB_PASSWORD=Secr3t! /bin/sh -c echo "build met \$DB_PASSWORD" > /trace.txt toont.

Uitleg. Het geheim zit in de image, leesbaar voor wie ze bezit. De juiste methode komt in lab 08.

Stap 9 — USER, zijn plaats, en wat het wordt in rootless-modus

```
printf 'FROM docker.io/library/eclipse-temurin:21-jre-alpine\nUSER 1000:1000\nWORKDIR /app\nRUN
mkdir /data\nCOPY api.jar /app/api.jar\nENTRYPOINT ["java","-jar","/app/api.jar"]\n' > D-user-
vroeg
podman build -f D-user-vroeg -t api-lab:user-vroeg . 2>&1 | grep -iE 'permission|error'
```

Observeer de mislukking: `mkdir: cannot create directory '/data': Permission denied`, en dan `Error: building at STEP "RUN mkdir /data": exit status 1`.

Zet `USER` als laatste:

```
printf 'FROM docker.io/library/eclipse-temurin:21-jre-alpine\nWORKDIR /app\nRUN mkdir /data && chown 1000:1000 /data\nCOPY --chown=1000:1000 api.jar /app/api.jar\nUSER 1000:1000\nENTRYPOINT ["java","-jar","/app/api.jar"]\n' > D-user-ok
podman build -q -f D-user-ok -t api-lab:user-ok .
podman run --rm --entrypoint id api-lab:user-ok
```

Observeer `uid=1000(...) gid=1000(1000) groups=1000(1000)` : de applicatie draait niet meer als root in de container.

Kijk nu wat dat **op de host** geeft:

```
podman run -d --name u api-lab:user-ok ; podman run -d --name r api-lab:1.0 ; sleep 1
podman top u user,huser,pid,hpid,comm
podman top r user,huser,pid,hpid,comm
podman rm -f -t 0 u r
```

Observeer:

USER	HUSER	PID	HPID	COMMAND	<- u: USER 1000 in de image
1000	100999	1	12929	java	
USER	HUSER	PID	HPID	COMMAND	<- r: root in de image
root	1000	1	13037	java	

Uitleg. `USER` geldt voor alles wat volgt, build **en** uitvoering; je zet het net vóór `ENTRYPOINT`, nadat je de bestanden hebt voorbereid. In rootless-modus is de "root" van de container al jouw gebruiker (`HUSER 1000`); de UID 1000 van de container wordt daarentegen geprojecteerd op `100999`, een UID uit het gereserveerde bereik van `/etc/subuid` dat *geen enkel* recht heeft op je host. `USER` blijft dus nuttig: het ontnemt de applicatie de root-privileges *binnen* de container (imagebestanden, poorten < 1024, `apk add`), en vooral: dezelfde image zal ooit onder Docker of Kubernetes draaien, waar root wel degelijk root is.

Opruimen

```
podman rm -f -t 0 api api2 2>/dev/null
podman rmi api-lab:1.0 api-lab:2.0 api-lab:2.1 api-lab:3.0 api-lab:3.1 \
    api-lab:cmd api-lab:shell-debian api-lab:shell-alpine \
    api-lab:script api-lab:script2 api-lab:arg api-lab:user-ok \
    api-lab:alles api-lab:alles2 2>/dev/null
podman images --format '{{.Repository}}:{{.Tag}}' | grep api-lab
podman rmi $(podman images --filter dangling=true -q) 2>/dev/null
rm -rf ~/labo-docker/04/node_modules ~/labo-docker/04/./gemeenschappelijk
```

Bewaar `~/labo-docker/04/api.jar` en `Api.java` : labs 05 tot 09 hergebruiken ze. Bewaar ook de images `eclipse-temurin:21-jre-alpine` en `eclipse-temurin:21-jdk`.

Wat je nu moet kunnen beweren

- De context bepaalt wat kopieerbaar is; met Podman wordt niets overgedragen, maar `.dockerignore` blijft onmisbaar voor wat in de image belandt.
- Een ongeldig gemaakte instructie maakt alle volgende ongeldig: de volgorde bepaalt de buildtijd.
- `EXPOSE` publiceert niets.
- `ENTRYPOINT` legt het programma vast, `CMD` geeft vervangbare argumenten, `--entrypoint` laat toe te debuggen.
- De *shell*-vorm kan de nette stop breken — en haar gedrag hangt af van de basisimage. Je hebt het gemeten: 0,14 s / code 143 tegenover 10 s / code 137.
- Een opstartscript moet eindigen met `exec`.
- Een `--build-arg` is zichtbaar in `podman history`.
- `USER` komt net vóór `ENTRYPOINT` — en behoudt zijn betekenis in rootless-modus.